SMALL-CAP GROWTH TRACKER · DOC 04

아키텍처 개요 (초안)

Software Architecture Overview

항목	내용
프로젝트	소형주 전문 수주잔고 & CAPEX 추적기
팀원	20225085 김수린 · 20232565 정수연
문서 버전	v0.2 (초안)
작성일	2026. 09. 21
개정일	2026. 10. 06 (v0.2 보완 — 부록 개정 이력 참조)
문서 상태	초안 — 구현 전 설계안 (검증 전)
선행 문서	프로젝트 계획서 (요구사항 FR · NFR 정의)
기준 커밋	TODO: 첫 구현 커밋 후 기입 (코드 인용의 기준점)

1. 문서 개요

> ※ 이 문서는 검증 전 초안입니다. 본문의 (추정) · (확인 필요) · TODO 표기는 구현 후 실제 코드와 데이터로 확인해 갱신하며, 확인된 내용에는 코드 위치(파일:줄번호) 또는 공시 접수번호를 근거로 남깁니다.

이 문서는 '소형주 전문 수주잔고 & CAPEX 추적기'의 전체 소프트웨어 구조를 설명합니다. 어떤 구성 요소가 있고, 데이터가 어떤 순서로 흐르며, 왜 이런 구조를 선택했는지를 정리하는 것이 목적입니다. 실제 폴더 배치와 협업 규칙은 별도 문서인 '22-저장소 구조'에서 다룹니다.

1.1 설계 원칙

2 인 팀이 한 학기 안에 완성할 수 있어야 하므로, 다음 다섯 가지 원칙을 기준으로 설계했습니다.

원칙	의미	이 프로젝트에서의 적용
단순성	규모에 맞지 않는 기술은 쓰지 않는다	서비스를 쪼개지 않고 저장소 1 개, 프로그램 2 개, DB 1 개로 구성
추적 가능성	화면의 모든 숫자는 출처를 따라갈 수 있어야 한다	모든 수치에 원천 공시 접수번호(rcept_no)를 함께 저장
재현성	같은 작업을 다시 해도 같은 결과가 나와야 한다	공시 원문 보관, 배치 재실행 시 중복 방지
분리	무거운 작업이 사용자 화면을 느리게 만들면 안 된다	데이터 가공(쓰기)과 화면 조회(읽기) 경로를 분리
최소 권한	각 프로그램은 필요한 권한만 가진다	배치 워커는 쓰기 계정, 웹 앱은 mart 읽기 전용 계정으로 접속. API 키는 저장소에 올리지 않음 (5.3 절)

1.2 관련 문서와 단일 출처

같은 내용을 여러 문서에 나눠 쓰면 한쪽만 고쳐져 서로 어긋나기 쉽습니다. 그래서 주제마다 **정본(正本) 문서**를 하나로 정하고, 이 문서에서는 구조와 연결 관계만 설명한 뒤 세부 내용은 정본을 참조합니다.

주제	정본 문서	이 문서에서 다루는 범위
요구사항(FR · NFR) 정의	프로젝트 계획서	ID 만 인용하고 8 장 추적표로 연결
폴더 배치 · 협업 규칙	22-저장소 구조	다루지 않음
테이블 컬럼 · 타입 상세	DB 스키마 문서 (작성 예정, TODO)	계층 구조와 예시 행만 제시 (5 장)
신호 판단 규칙의 수식 · 임계값	domain 모듈 코드 + 단위 테스트	대표 예시만 제시 (7 장)

문서 파일은 저장소의 `docs/` 아래에 모으고, 실제로 실행되는 코드(배치 워커 · 웹 앱 진입점, 의존성 · 설정 파일)는 저장소 루트에 그대로 둡니다. 배포 플랫폼이 루트를 기준으로 앱을 실행하므로,

정리한다고 이 파일들을 옮기면 서비스가 동작하지 않을 수 있습니다. 세부 배치는 22-저장소 구조를 따릅니다.

2. 아키텍처 스타일 선택

본 시스템은 **모듈형 모놀리스(Modular Monolith)**를 기본 골격으로 하고, 여기에 **배치 파이프라인(Pipe-Filter)**과 **경량 CQRS** 를 결합한 구조입니다.

스타일	쉬운 설명	적용 위치
모듈형 모놀리스	하나의 저장소, 하나의 코드베이스 안에서 기능별로 폴더(모듈)를 명확히 나누는 방식	프로젝트 전체
배치 파이프라인 (Pipe-Filter)	데이터가 여러 처리 단계를 차례로 통과하며 가공되는 방식. 공장의 컨베이어 벨트와 비슷함	수집 → 추출 → 계산 → 요약
경량 CQRS	데이터를 '쓰는 쪽'과 '읽는 쪽'을 분리. 무거운 계산은 미리 해 두고 화면은 결과만 읽음	배치 워커(쓰기) / 웹 앱(읽기)
계층형(Layered)	코드를 역할별 층으로 나눠 위층이 아래층만 호출하게 하는 방식	웹 앱 내부, DB 접근 코드

2.1 이 구조를 선택한 이유

이 서비스의 데이터는 **쓰기는 하루 한 번 무겁게, 읽기는 하루 종일 가볍게** 일어난다는 뚜렷한 특징이 있습니다. 매일 18:30 에 수천 건의 공시를 LLM 으로 분석하는 작업은 수십 분이 걸릴 수 있지만, 사용자가 랭킹 화면을 여는 것은 이미 계산된 표를 보여주기만 하면 됩니다. 두 작업을 분리하면 LLM 분석이 아무리 오래 걸려도 화면 응답 속도에는 영향이 없습니다.

2.2 고려했으나 제외한 선택지

선택지	제외 이유
마이크로서비스	서비스 간 통신, 배포, 모니터링 부담이 커서 2 인 팀과 2,400 개 종목 규모에 비해 과도함
메시지 브로커 (Kafka 등)	하루 한 번 실행되는 배치에는 실시간 이벤트 전달이 필요 없음
별도 벡터 DB	보고서의 '사업의 내용' 섹션만 잘라 LLM 에 전달하면 충분함. 필요 시 PostgreSQL 확장(pgvector)으로 대체 가능
워크플로우 엔진 (Airflow 등)	파이프라인이 4 단계로 단순하여 cron 또는 APScheduler 로 충분함

3. 전체 구성

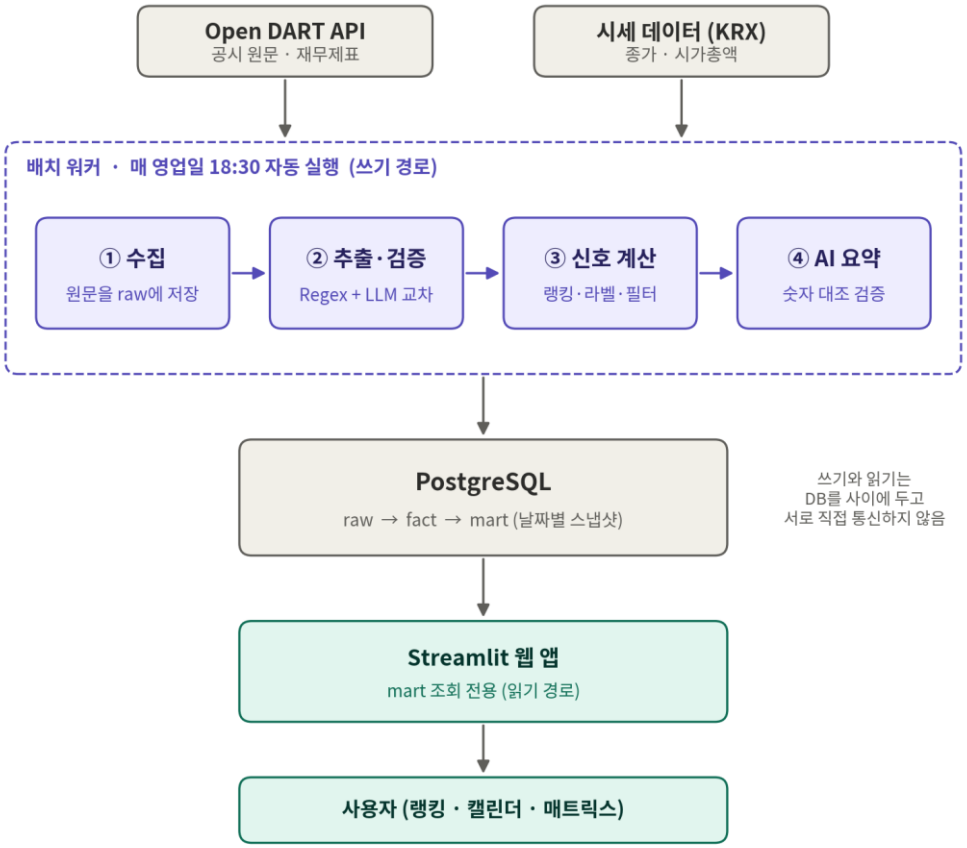


그림 1. 전체 아키텍처 구성도

구성 요소	역할	실행 시점
배치 워커	외부 데이터 수집, 수치 추출, 신호 계산, AI 요약을 수행하고 결과를 DB 에 저장	매 영업일 18:30 실행 후 종료
PostgreSQL	원문부터 최종 결과까지 모든 데이터를 저장. 배치 워커와 웹 앱의 연결 지점	상시 가동
Streamlit 웹 앱	DB 의 결과 테이블을 읽어 랭킹, 캘린더, 매트릭스 화면으로 표시	상시 가동 (사용자 접속 시 조회)
외부: Open DART API	정기보고서, 시설투자 공시 원문과 재무제표 제공	배치 수집 단계에서 호출
외부: 시세 데이터 (KRX)	종가, 시가총액 제공 (DART 에는 없는 정보)	배치 수집 단계에서 호출
외부: LLM API	비정형 텍스트에서 수치 추출, 투자 요약 문장 생성	배치 추출·요약 단계에서 호출

> **핵심 포인트**
> 배치 워커와 웹 앱은 서로 직접 통신하지 않습니다. 배치 워커는 DB 에 **쓰기만** 하고, 웹 앱은 DB 에서 **읽기만** 합니다. DB 가 둘 사이의 약속 장소 역할을 합니다.

4. 배치 파이프라인 상세

배치 워커는 네 단계로 구성되며, 각 단계는 **DB 테이블을 사이에 두고** 연결됩니다. 한 단계가 끝나면 결과를 테이블에 저장하고, 다음 단계는 그 테이블을 읽어서 시작합니다.

단계	입력	처리 내용	출력	관련 요구사항
① 수집	Open DART, KRX	신규 공시 원문, 재무제표, 종가·시가총액을 받아 그대로 저장	raw 테이블	FR-01, NFR-01
② 추출·검증	raw 테이블	Regex 와 LLM 으로 수주잔고·가동률·완공일을 추출하고 두 결과를 교차 검증	fact 테이블	FR-02, NFR-04
③ 신호 계산	fact 테이블	수주잔고/시총 비율, 턴어라운드 태그, 듀얼 매트릭스 라벨, 리스크 필터 계산	mart 테이블	FR-03~05, 07, 08
④ AI 요약	mart 테이블	근거 수치만 LLM 에 전달해 1~2 문장 요약 생성 후 숫자 대조 검증	mart 테이블	FR-06, NFR-02

4.1 원문 보관과 재파싱

수집 단계는 공시 원문을 가공하지 않고 `raw` 테이블에 그대로 저장한 뒤 끝납니다. 숫자를 뽑는 일은 추출 단계가 따로 맡습니다.

이렇게 나누면 LLM 프롬프트를 개선했을 때 **DART 를 다시 호출하지 않고 저장된 원문으로 추출 단계만 다시 실행(재파싱)**할 수 있습니다. Open DART 는 API 키당 하루 호출 한도가 있으므로, 2,400 개 종목의 보고서를 매번 새로 받아오는 것은 비효율적입니다. 또한 로드맵 Phase 1(파싱 정확도 개선)을 원문 재수집 없이 진행할 수 있습니다.

> **쉽게 말하면**
> 교과서를 빌려서 요약노트만 만들고 반납하면, 요약이 틀렸을 때 다시 빌려야 합니다. 교과서를 옆에 두면 다시 펴 보기만 하면 됩니다. `raw` 테이블이 바로 옆에 둔 교과서입니다.

4.2 중복 방지 (멱등성)

DART 의 모든 공시에는 고유한 **접수번호(rcept_no)**가 부여됩니다. 이를 `raw` 테이블의 고유 키로 지정하여, 같은 공시를 두 번 저장하려 하면 DB 가 자동으로 건너뛰도록 합니다.

```
INSERT INTO raw_disclosures (rcept_no, corp_code, report_type, content)
VALUES ('20260921000123', '00123456', '반기보고서', '...원문...')
ON CONFLICT (rcept_no) DO NOTHING;   -- 이미 있으면 무시
```

따라서 배치가 중간에 실패해도 처음부터 다시 실행하면 빠진 부분만 채워집니다. 이처럼 여러 번 실행해도 결과가 같은 성질을 **멱등성(Idempotency)**이라고 합니다.

4.3 추출 단계의 교차 검증

추출은 'Regex 먼저, LLM 보조, 교차 검증' 순서로 진행합니다.

1. 표 형태로 정형화된 수치는 **정규표현식(Regex)**으로 먼저 추출합니다. 빠르고 정확하지만 표현이 조금만 달라져도 놓칠 수 있습니다.
2. 서술형 문단은 **LLM** 이 JSON 형식으로 추출합니다. 표현이 달라도 이해하지만 가끔 틀릴 수 있습니다.
3. 두 결과가 일치하면 신뢰하고, 다르거나 단위(억·백만·천원)가 애매하면 `review_flag` 를 설정해 랭킹에서 제외하고 수기 검증 대상으로 넘깁니다(NFR-04).

4.4 AI 요약의 환각 방지

NFR-02(원천 DB 수치만 사용)를 프롬프트 지시만으로 보장하기는 어렵습니다. 그래서 생성 후 **사후 검증 단계**를 둡니다. 요약 문장에서 숫자를 모두 뽑아, 각 숫자가 LLM 에 입력으로 넘긴 값 목록에 있는지 확인합니다. 입력에 없던 숫자가 하나라도 있으면 재생성하거나 사전에 정의한 템플릿 문장으로 대체합니다.

```
def check_grounding(summary: str, allowed: set[str]) -> bool:
    found = re.findall(r"\d+(?:\.\d+)?", summary)
    return all(n in allowed for n in found)
```

4.5 LLM 호출 계층과 재현성

추출(②)과 요약(④) 단계는 LLM API 를 직접 부르지 않고 공통 모듈 **llm_client** 를 거칩니다. 단계 코드는 어떤 모델을 쓰는지 모른 채 정해진 함수만 호출하므로, 모델이나 공급사를 바꿔도 llm_client 설정만 수정하면 됩니다. LLM 모델 라인업은 분기 단위로 바뀌는 경우가 많아, 교체 가능성을 처음부터 열어 두는 편이 안전합니다.

```
class LLMClient(Protocol):
    def extract_json(self, text: str, schema: dict) -> dict: ...
    def summarize(self, facts: dict) -> str: ...
```

또한 같은 결과를 다시 만들 수 있도록(재현성), LLM 이 만든 모든 값에는 아래 실행 정보를 함께 저장합니다. 금융 정보를 다루는 AI 기능은 사용 모델 · 프롬프트 · 설정이 기록되어 있어야 결과를 설명하고 검증할 수 있습니다.

기록 항목	예시	용도
`model_id`	확정된 모델명 (TODO)	모델 교체 전후 결과 비교
`prompt_version`	extract_v3	프롬프트 개선 효과 측정, 재파싱 대상 선정
`temperature`	0	같은 입력에 대한 출력 흔들림 최소화
`input_hash`	입력 텍스트의 SHA-256	같은 입력의 중복 호출 방지 (비용 절감)

이 정보는 fact_metrics 의 `method` 컬럼을 보완하는 형태로 저장하며, 구체적인 컬럼 정의는 DB 스키마 문서에서 확정합니다(TODO).

4.6 배치 실행 기록과 실패 처리

NFR-04 의 오류 로그를 파일에만 남기면 웹 앱은 오늘 배치가 성공했는지 알 수 없습니다. 그래서 배치 실행 결과를 **batch_runs** 테이블에 단계별로 기록합니다.

컬럼	의미

컬럼	의미
`run_id`	실행 고유 번호
`run_date` · `stage`	기준 날짜와 단계 (① 수집 ~ ④ AI 요약)
`status`	running / success / failed
`processed` · `flagged`	처리 건수와 review_flag 설정 건수
`error_message`	실패 원인 요약
`started_at` · `finished_at`	단계별 소요 시간 추적 (NFR-01 지연 감지)

1. 단계가 실패하면 그 단계에서 멈추고, 앞 단계 결과는 테이블에 그대로 남습니다. 다시 실행하면 실패한 단계부터 이어서 진행합니다(4.2 절 멱등성).
2. 웹 앱은 가장 최근에 성공한 `signal_date` 를 보여 주고, 화면 상단에 데이터 기준 시각을 표시합니다(6.1 절).
3. 오늘 배치가 실패했다면 '전일 기준 데이터' 안내를 함께 띄워, 사용자가 오래된 신호를 최신으로 오해하지 않게 합니다.

4.7 초기 적재와 일일 증분

서비스를 처음 시작할 때는 약 2,400 개 종목의 과거 보고서를 한꺼번에 모아야 하고(**초기 적재**), 그 뒤에는 매일 새로 나온 공시만 처리하면 됩니다(**일일 증분**). 두 작업은 처리량이 크게 다르므로, 같은 파이프라인 코드를 쓰되 실행 모드만 나눕니다.

```
python -m batch run --mode backfill --from 2025-01-01   # 초기 적재 (여러 날에 나눠 실행)
python -m batch run --mode daily                         # 매 영업일 18:30
```

초기 적재는 DART 일일 호출 한도 안에서 며칠에 걸쳐 나눠 실행하고, 멱등성 덕분에 중간에 끊겨도 이어서 받을 수 있습니다. 초기 적재와 일일 처리의 LLM 비용 추정은 9.1 절에서 다룹니다.

5. 데이터 계층 설계

DB 테이블은 가공 단계에 따라 세 층으로 나눕니다.

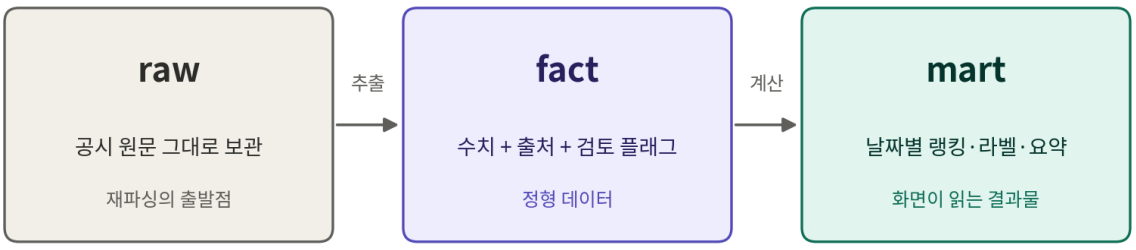


그림 2. 데이터 계층 구조

계층	저장 내용	역할	주요 테이블 (예정)
raw	공시 원문, 재무제표 JSON, 일별 시세	원천 보관, 재처리의 출발점	raw_disclosures, raw_financials, raw_prices

계층	저장 내용	역할	주요 테이블 (예정)
fact	수주잔고(원), 가동률(%), 완공 예정일 등 정형 수치	수치마다 출처 공시번호, 추출 방식, 검토 플래그를 함께 저장	fact_metrics, fact_capex_schedule
mart	날짜별 랭킹, 매트릭스 라벨, 리스크 필터 결과, AI 요약	화면이 바로 읽는 최종 결과물	mart_daily_signal, mart_briefing

5.1 데이터가 계층을 통과하는 예시

A 테크솔루션의 반기보고서 하나가 세 계층을 거치는 과정입니다.

raw_disclosures

rcept_no	corp_name	report_type	content
20260814000321	A 테크솔루션	반기보고서	"...당사의 수주잔고는 420,000 백만원이며..." (원문 전체)

fact_metrics

corp_name	metric	value	unit	source_rcept_no	method	review_flag
A 테크솔루션	backlog	420000000000	원	20260814000321	regex+llm 일치	false
A 테크솔루션	utilization	87.0	%	20260814000321	llm	false

mart_daily_signal

signal_date	corp_name	backlog_ratio	matrix_label	tags
2026-09-21	A 테크솔루션	350%	Turnaround Star	턴어라운드

5.2 날짜별 스냅샷

`mart` 테이블은 매일 덮어쓰지 않고 `signal_date` 컬럼을 두어 날짜별로 쌓습니다. 이렇게 하면 '그 시점에 시스템이 어떤 신호를 냈는가'가 자동으로 기록되어, 로드맵 Phase 2(과거 공시 기반 백테스팅)의 검증 데이터로 바로 활용할 수 있습니다.

> **PostgreSQL 이란**
> 오픈소스 관계형 데이터베이스입니다. 엑셀 시트처럼 표(테이블) 형태로 데이터를 저장하지만, 컬럼마다 데이터 타입이 엄격히 정해져 있고 여러 프로그램이 동시에 읽고 써도 데이터가 꼬이지 않습니다. 공시 원문 같은 JSON 데이터를 저장·검색하는 기능(jsonb)이 강하고, Supabase·Neon 등에서 무료 클라우드 DB 로 바로 사용할 수 있어 선택했습니다.

5.3 DB 계정 분리와 비밀정보 관리

경량 CQRS 구조를 코드 규칙뿐 아니라 **DB 권한으로도 강제**합니다. '웹 앱은 읽기만 한다'를 코드 규칙으로만 정해 두면 실수로 쓰기 코드가 들어가도 막을 수 없지만, 웹 앱이 읽기 전용 계정으로 접속하면 DB 가 직접 거부합니다.

계정	사용 주체	권한
`batch_writer`	배치 워커	raw · fact · mart · batch_runs 읽기 · 쓰기
`app_reader`	Streamlit 웹 앱	mart · batch_runs 읽기(SELECT)만 허용
관리자 계정	팀원 (스키마 변경 시)	전체 권한 — 평소 프로그램 실행에는 사용하지 않음

```
GRANT SELECT ON mart_daily_signal, mart_briefing, batch_runs TO app_reader;
-- raw · fact 테이블과 쓰기 권한은 부여하지 않음
```

Open DART API 키, LLM API 키, DB 접속 정보는 코드에 직접 쓰지 않습니다. 로컬에서는 `.env` 파일(`.gitignore` 에 등록)로, 배포 환경에서는 플랫폼의 비밀값 설정(Streamlit secrets 등)으로 주입합니다. 저장소에는 키 이름만 적은 `.env.example` 만 올립니다.

> **쉽게 말하면**
> 도서관에서 열람실 출입증과 서고 관리자 열쇠를 따로 나눠 주는 것과 같습니다. 열람실 출입증(app_reader)으로는 책을 읽을 수만 있고, 실수로라도 책을 고치거나 버릴 수 없습니다.

6. 화면 조회 흐름

사용자가 랭킹 화면을 열 때의 처리 순서입니다.

1. 사용자가 웹 앱의 '수주잔고 랭킹' 페이지에 접속합니다.
2. 화면 코드가 조회 함수(repository)에 '2026-09-21 기준 상위 종목'을 요청합니다.
3. repository 가 `mart_daily_signal` 에서 해당 날짜 데이터를 SQL 로 조회합니다.
4. 조회 결과를 표와 막대 차트로 그려 사용자에게 보여줍니다.

계산은 이미 배치에서 끝났으므로 화면은 단순 조회만 수행합니다. 따라서 NFR-03(3 초 이내 응답)을 별도의 최적화 없이 충족할 수 있습니다.

```
SELECT corp_name, backlog_ratio, matrix_label
FROM mart_daily_signal
WHERE signal_date = '2026-09-21' AND review_flag = false
ORDER BY backlog_ratio DESC
LIMIT 5;
```

6.1 모든 화면에 공통으로 표시할 요소

랭킹 · 캘린더 · 매트릭스 화면은 공통 레이아웃을 공유하며, 다음 세 가지를 항상 함께 보여 줍니다.

- **데이터 기준 시각**: batch_runs 에서 마지막으로 성공한 배치의 완료 시각 (4.6 절)
- **근거 공시 링크**: 각 수치 옆에 접수번호(rcept_no)로 만든 DART 원문 링크를 달아 사용자가 출처를 직접 확인하게 합니다. 설계 원칙 '추적 가능성'을 화면에서 구현하는 부분입니다.

- **투자 면책 문구**: 금융 정보 서비스이므로 모든 페이지 하단에 고정합니다. 예: "본 서비스의 정보는 공시 데이터를 가공한 참고 자료이며 투자 권유가 아닙니다. 투자 판단과 그 결과에 대한 책임은 이용자 본인에게 있습니다."

```
https://dart.fss.or.kr/dsaf001/main.do?rcept_no={rcept_no}   # 근거 공시 링크 형식
```

또한 mart 데이터는 하루 한 번만 바뀌므로, 조회 결과를 Streamlit 캐시(`st.cache_data`)에 담아 두면 여러 사용자가 같은 화면을 열어도 DB 조회가 반복되지 않습니다. 캐시 유효 시간(ttl)을 1 시간 정도로 두면 배치 완료 후 늦어도 1 시간 안에 새 데이터가 반영됩니다.

7. 도메인 로직 분리

투자 신호를 판단하는 규칙은 DB 나 외부 API 에 의존하지 않는 **순수 함수(Pure Function)**로 `domain` 모듈에 모읍니다. 순수 함수는 같은 입력에 항상 같은 결과를 돌려주는 함수입니다.

```
def backlog_ratio(backlog: float, market_cap: float) -> float:
    return backlog / market_cap * 100

def is_turnaround(utilization: float, has_new_capex: bool) -> bool:
    return utilization >= 85 and has_new_capex
```

대상 규칙은 수주잔고/시총 비율 계산, 턴어라운드 태그 부여, 듀얼 매트릭스 4 분면 라벨링, 가짜 일감 필터(FR-07), 자금 조달 필터(FR-08)입니다. 이렇게 분리하면 다음과 같은 이점이 있습니다.

- DB 없이도 단위 테스트가 가능합니다. 예: `backlog_ratio(4200, 1200) == 350`
- 백테스팅 시 과거 데이터에 같은 함수를 그대로 재사용할 수 있습니다.
- 화면 기술을 바꾸더라도(예: Streamlit 에서 React 로) 핵심 로직은 수정할 필요가 없습니다.

7.1 테스트와 품질 지표

테스트는 세 층으로 나눕니다. 위쪽 행일수록 빠르고 자주 실행합니다.

테스트	대상	방법	실행 시점
단위 테스트	domain 순수 함수	pytest 로 DB · API 없이 실행	코드를 수정할 때마다
추출 회귀 테스트	② 추출 · 검증	저장해 둔 공시 원문 샘플(픽스처)의 추출 결과를 사람이 확인한 정답값과 비교	프롬프트 · 정규식 · 모델 변경 시
근거 검증 테스트	④ AI 요약	`check_grounding` 이 입력에 없는 숫자를 잡아내는지 확인	요약 로직 변경 시

추출 회귀 테스트를 위해 다양한 표현과 단위(억 · 백만원 · 천원)가 섞인 공시 원문을 골라 **정답셋(golden set)**을 만듭니다. 규모는 30~50 건 수준에서 시작합니다(추정). 이 정답셋으로 다음 지표를 측정해, 프롬프트나 모델을 바꿀 때마다 비교합니다.

- **추출 정확도**: 정답과 일치한 수치의 비율 (단위까지 맞아야 정답)
- **review_flag 비율**: 수기 검증으로 넘어간 비율. 너무 높으면 자동화 효과가 떨어지고, 너무 낮으면 검증이 느슨하다는 신호
- **근거 위반률**: 요약 재생성 또는 템플릿 대체가 발생한 비율 (NFR-02)

정답셋 원문과 측정 결과는 테스트 자료로 함께 보관해, 로드맵 Phase 1(파싱 정확도 개선)의 성과를 숫자로 보여 주는 근거로 사용합니다.

8. 요구사항 추적표

계획서의 요구사항이 아키텍처의 어느 부분에서 구현되는지 정리했습니다.

ID	요구사항	구현 위치	구현 방식
FR-01	공시 및 재무 데이터 파이프라인	배치 ① 수집	DART·KRX 수집, 원문 raw 저장
FR-02	비정형 텍스트 구조화 DB 적재	배치 ② 추출·검증	Regex + LLM 교차 추출 → fact
FR-03	수주잔고/시총 비율 스크리닝	domain + 웹 앱	비율 계산 후 mart 저장, 랭킹 화면
FR-04	CAPEX 턴어라운드 캘린더	domain + 웹 앱	완공·양산 일정 fact → 캘린더 화면
FR-05	Trailing vs Leading 듀얼 매트릭스	domain + 웹 앱	4 분면 라벨링 순수 함수
FR-06	AI 기반 자연어 투자 요약	배치 ④ AI 요약	근거 수치만 LLM 에 전달
FR-07	가짜 일감 필터	domain	매출채권 회전일수·미수금 추이 점검
FR-08	자금 조달 필터	domain	CB·BW·유상증자 이력 점검
NFR-01	매 영업일 18:30 배치	스케줄러	cron 또는 APScheduler
NFR-02	환각 방지	배치 ④ AI 요약	생성 후 숫자 대조 검증
NFR-03	3 초 이내 응답	경량 CQRS 구조	사전 계산된 mart 단순 조회
NFR-04	예외 처리·수기 검증	배치 ② 추출·검증	review_flag, 오류 로그
제안-01	투자정보 면책 고지	웹 앱 공통 레이아웃	모든 페이지 하단 고정 문구 (6.1 절)
제안-02	API 키 · DB 계정 보호	설정 · DB 권한	.env / secrets 주입, 읽기 전용 계정 (5.3 절)
제안-03	배치 상태 표시	batch_runs + 웹 앱	데이터 기준 시각 · 실패 안내 (4.6 절)

※ '제안' 항목은 v0.2 아키텍처 검토 중 도출한 것으로, 계획서 요구사항 개정 시 정식 ID 를 부여합니다.

9. 기술 스택

영역	기술	선택 이유
언어	Python 3.11+	배치, 데이터 분석, 웹 화면을 한 언어로 통일
데이터베이스	PostgreSQL (Supabase 무료 티어)	JSON 저장 기능, 팀원 공동 접근 용이
스케줄러	cron / APScheduler	4 단계 배치에 충분한 단순한 도구
웹 화면	Streamlit	Python 만으로 화면 구현 가능, 개발 속도 확보
외부 데이터	Open DART API, pykrx	공시·재무제표 / 종가·시가총액

영역	기술	선택 이유
LLM	LLM API (모델 추후 확정) — 9.1 절 참조	비정형 텍스트 추출, 요약 생성
테스트	pytest	domain 순수 함수 단위 테스트
형상관리	Git, GitHub	버전 관리 및 협업 (22-저장소 구조 문서 참조)
설정 · 비밀정보	python-dotenv, Streamlit secrets	API 키 · DB 접속 정보를 코드와 분리 (5.3 절)
AI 코딩 도구	AGENTS.md 규칙 파일 + 팀원별 선택 도구	아키텍처 규칙을 AI 코딩 도구가 따르도록 고정 (9.2 절)

9.1 LLM 모델 선정 기준과 비용 추정

LLM 모델은 아직 확정하지 않았지만(TODO), 다음 세 가지 기준으로 고릅니다.

1. **데이터를 어디로 보내도 되는가**: 입력은 공개된 공시 원문과 그 수치뿐이며 개인정보가 없으므로, 외부 LLM API 로 보내는 데 제약이 적습니다.
2. **무엇으로 결과를 검증하는가**: 공개 벤치마크 점수보다 우리 데이터에서의 정확도가 기준입니다. 후보 모델을 7.1 절 정답셋으로 직접 측정해 비교합니다.
3. **한 달 비용 상한을 얼마로 묶는가**: 아래 추정치로 월 상한을 정하고, API 콘솔의 사용량 한도 기능으로 넘지 않게 합니다.

기본안은 **역할별 모델 분리**입니다. 호출량이 많은 추출(②)에는 저가 모델을, 상위 종목에만 실행되는 요약(④)에는 문장 품질이 좋은 중급 모델을 씁니다. 아래 표는 후보 모델의 단가와, 공시 1 건당 입력 30,000 토큰 · 출력 1,000 토큰을 가정한 1,000 건당 비용입니다.

후보 모델	입력 $/1M	출력 $/1M	1,000 건당 (추정)	검토 용도
Gemini 2.5 Flash-Lite	0.10	0.40	약 $3.4	추출 (최저가)
GPT-5.6 Luna	0.20	1.20	약 $7.2	추출
Claude Haiku 4.5	1.00	5.00	약 $35	추출 · 요약
Gemini 3.6 Flash	1.50	7.50	약 $52.5	요약
Claude Sonnet 5	2.00	10.00	약 $70	요약 (고품질)

단가 출처: 강의자료「AI 코딩 터미널 · 에디터 10 종 완전 비교」API 요금 비교 (2026 년 9 월, 미국 달러 정가). 가격은 변동이 잦으므로 확정 전 각 공식 가격표에서 재확인합니다. 1 건당 토큰 수는 (추정)이며, 실제 '사업의 내용' 섹션 길이를 측정해 보정해야 합니다.

규모 감을 잡기 위한 계산입니다. 2,400 개 종목의 정기보고서(연 4 회)만 처리해도 연 약 9,600 건이므로, 저가 추출 모델은 연 30~70 달러, Haiku · Sonnet 급은 연 300~700 달러 수준입니다(추정). 초기 적재로 과거 1 년치를 처리하면 비슷한 금액이 한 번 더 듭니다.

> **비용을 더 줄이는 방법**
>
> ① 배치 할인: 즉시 응답이 필요 없는 작업은 배치 API 로 보내면 50% 할인 요율이 적용됩니다(OpenAI · Google 등). 다만 결과가 늦게(최대 24 시간) 돌아올 수 있으므로 초기 적재처럼 급하지 않은 작업에 우선 적용합니다.

> ② 입력 캐시: 매번 같은 지시문(시스템 프롬프트)을 앞에 붙이므로, 캐시된 입력 할인(대략 정가의 10%)을 받을 수 있게 프롬프트 앞부분을 고정합니다.
>
> ③ 긴 컨텍스트 할증 회피: 일부 모델은 요청이 200K 토큰을 넘으면 요청 전체가 2 배 요율로 바뀝니다. '사업의 내용' 섹션만 잘라 보내는 현재 설계를 유지합니다.
>
> ④ 중복 호출 방지: input_hash 가 같은 입력은 다시 호출하지 않습니다(4.5 절).

9.2 개발 도구: AI 코딩 보조 활용

2 인 팀의 개발 속도를 높이기 위해 AI 코딩 도구(Claude Code, Cursor, GitHub Copilot 등)를 활용합니다. 어떤 도구를 쓸지는 팀원별로 달라도 되지만, 이 문서의 아키텍처 규칙은 저장소 루트의 규칙 파일 **AGENTS.md** 에 적어 모든 도구가 같은 규칙을 따르게 합니다. AGENTS.md 는 여러 AI 코딩 도구가 공통으로 읽는 형식으로 수렴하는 추세이며, Claude Code 처럼 전용 파일(CLAUDE.md)을 쓰는 도구는 그 파일에서 AGENTS.md 를 불러오도록 연결합니다.

```
# AGENTS.md — 아키텍처 규칙 (04-아키텍처 개요 기준)
- web/ 코드는 DB 에 쓰지 않는다 (app_reader 계정, SELECT 만)
- domain/ 은 db · 외부 API · LLM 모듈을 import 하지 않는다 (순수 함수)
- LLM 호출은 반드시 llm_client 를 거친다
- 화면에 표시하는 모든 수치는 source_rcept_no 를 함께 조회한다
- domain 함수를 추가 · 수정하면 pytest 테스트를 함께 작성한다
- API 키 · 비밀번호를 코드나 커밋에 넣지 않는다
```

도구가 지원하는 경우 다음 기능도 함께 씁니다.

- **훅(Hooks) · pre-commit**: 파일 수정이나 커밋 직전에 pytest 를 자동 실행해, AI 가 domain 규칙을 깨뜨리면 바로 알 수 있게 합니다.
- **MCP 연결**: 개발용 DB 를 app_reader 같은 읽기 전용 계정으로 AI 도구에 연결하면, AI 가 실제 테이블 구조를 보고 SQL 을 작성할 수 있습니다. 쓰기 권한이 있는 계정은 연결하지 않습니다.
- **계획 모드(Plan Mode)**: 여러 파일에 걸친 수정은 먼저 계획만 받아 검토한 뒤 실행해, 모듈 경계를 넘는 변경을 미리 걸러 냅니다.

규칙 파일과 MCP 설정은 도구를 바꿔도 거의 그대로 옮겨 쓸 수 있으므로 특정 도구에 묶이지 않습니다. AI 가 작성한 코드도 사람이 작성한 코드와 똑같이 7.1 절 테스트를 통과해야 병합합니다.

10. 확장 계획

초기에는 Streamlit 이 DB 를 직접 조회하는 단순한 구조로 시작합니다. 서비스 완성도를 높여야 할 경우, 아래와 같이 확장할 수 있도록 설계했습니다.

단계	변경 내용	영향 범위
확장 1	FastAPI 서버를 추가해 조회 기능을 API 로 제공 (router → service → repository 3 계층)	api 폴더 추가, domain·db 코드 재사용
확장 2	React(Next.js) 프론트엔드로 화면 교체	web 폴더 추가, API 만 호출
확장 3	mart 스냅샷을 활용한 백테스팅 모듈 추가 (로드맵 Phase 2)	domain 함수 재사용
확장 4	LLM 공급사 교체 · 병행 (예: 추출과 요약에 서로 다른 공급사 모델 사용)	llm_client 구현체만 추가, 파이프라인 단계 코드는 변경 없음

11. 미결 사항 및 리스크

항목	내용	대응 방향
시세 데이터 출처	Open DART 에는 주가·시가총액이 없음	pykrx 또는 KRX 정보데이터시스템 사용 여부 확정 필요
DART 호출 한도	API 키당 일일 호출 한도 존재	원문 보관으로 재호출 최소화, 신규 공시만 수집
공시 마감 시각	18:00 마감 가정의 정확성 (확인 필요)	실제 공시 접수 시각 분포를 확인하고, 다음 날 오전 보정 배치 검토
LLM 비용	종목 수 × 보고서 길이에 비례해 비용 증가	'사업의 내용' 섹션만 잘라 전달, 변경된 공시만 처리
단위 오인식	억·백만원·천원 혼재	Regex·LLM 교차 검증과 review_flag 로 랭킹 왜곡 방지
배치 실행 환경	Streamlit 호스팅은 화면 서빙용이라 정기 배치 실행에 적합하지 않으며, 배치를 어디서 돌릴지 미정 (TODO)	개인 PC cron, GitHub Actions 예약 실행, 소형 클라우드 VM 중 선택. GitHub Actions 예약은 UTC 기준(18:30 KST = 09:30 UTC)이며 실행이 지연될 수 있음
공시 시즌 쏠림	분기 · 반기 · 사업보고서 제출 마감일 전후에는 하루 공시량이 평소보다 크게 늘어 배치 시간과 LLM 비용이 급증 (추정)	시즌에는 처리 대상을 우선순위(시총 · 업종)로 나눠 다음 날까지 분산, 배치 API 활용 (9.1 절)
LLM 모델 변경 · 단종	모델 라인업이 자주 바뀌어 같은 프롬프트도 결과가 달라질 수 있음	model_id · prompt_version 기록(4.5 절), 교체 전 정답셋 회귀 테스트(7.1 절)
비밀정보 유출	API 키 · DB 비밀번호가 커밋에 포함될 위험	.gitignore 와 .env.example 사용, 유출 시 즉시 키 재발급 (5.3 절)
투자 오인	사용자가 신호를 매수 추천으로 받아들일 위험	면책 문구 고정, 근거 공시 링크 제공 (6.1 절)

부록. 개정 이력

버전	날짜	변경 내용
v0.1	2026. 09. 21	초안 작성
v0.2	2026. 10. 06	최소 권한 원칙(1.1), 관련 문서와 단일 출처(1.2), LLM 호출 계층 · 배치 실행 기록 · 초기 적재(4.5~4.7), DB 계정 분리(5.3), 화면 공통 요소(6.1), 테스트와 품질 지표(7.1), 제안 요구사항(8), LLM 선정 기준과 비용 추정 · AI 코딩 도구 활용(9.1~9.2), 확장 4(10), 리스크 5 건(11) 추가